

01_04_red_neuronal

May 7, 2026

1 Ejercicio realizado por:

1.1 Hernández Valdivia Adair

1.2 Ortiz Barron Moises Sebastian

2 Red neuronal multicapa en PyTorch

En este notebook veremos como implementar una red neuronal usando PyTorch. En particular, implementaremos un perceptrón multicapa (MLP por sus siglas en inglés) para la clasificación de dígitos del conjunto de datos MNIST. Total de puntos: 2

@juanlrvng

```
[1]: # Primero llamamos a los paquetes necesarios
%matplotlib inline
%config InlineBackend.figure_format = 'retina'

import helper
import matplotlib.pyplot as plt

import numpy as np
import torch
from torchvision import datasets, transforms
```

2.1 Conjunto de datos

Para la práctica necesitaremos un conjunto de datos (dataset). Afortunadamente el paquete **torchvision** provee diversos conjuntos de datos de ejemplo. En este ejercicio, utilizaremos MNIST, el cual contiene ejemplos de letras escritas a mano. El siguiente código lee el conjunto de datos y lo separa en un conjunto de entrenando y uno de prueba.

En PyTorch, las clases **Dataset** y **DataLoader** son fundamentales para gestionar y procesar datos de manera eficiente en aplicaciones de aprendizaje profundo. Estas clases simplifican el manejo de datos, asegurando que los datos se presenten en lotes, se mezclen aleatoriamente y se procesen de manera escalable, incluso para grandes conjuntos de datos.

La clase **Dataset** es una abstracción que define cómo se accede y manipula un conjunto de datos. Sirve como base para crear datasets personalizados o usar datasets predefinidos de PyTorch (como los de **torchvision**).

El `DataLoader` se encarga de cargar los datos desde un `Dataset`, gestionando el muestreo, el manejo por lotes y el paralelismo. Es especialmente útil para trabajar con grandes conjuntos de datos o para preparar datos en lotes para entrenamiento. Parámetros clave de `DataLoader`: 1. **dataset**: El objeto `Dataset` que contiene los datos. 2. **batch_size**: Cantidad de muestras por lote. Si no se especifica, se usa `batch_size=1`. 3. **shuffle**: Si es `True`, mezcla los datos aleatoriamente. 4. **num_workers**: Número de procesos secundarios para cargar datos en paralelo. 5. **drop_last**: Si es `True`, descarta el último lote si no contiene suficientes datos.

```
[2]: # Generamos una transformación para convertir las matrices a tensores y
      ↪normalizar el conjunto de datos
transform = transforms.Compose([transforms.ToTensor(),
                                transforms.Normalize([0.5], [0.5])
                                ])

# Descargamos el conjunto de datos de entrenamiento
trainset = datasets.MNIST('MNIST_data/', download=True, train=True,
    ↪transform=transform)
# Cargamos el conjunto
batch_size=64
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
    ↪shuffle=True)

# Descargamos y cargamos el conjunto de prueba
testset = datasets.MNIST('MNIST_data/', download=True, train=False,
    ↪transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
    ↪shuffle=True)
```

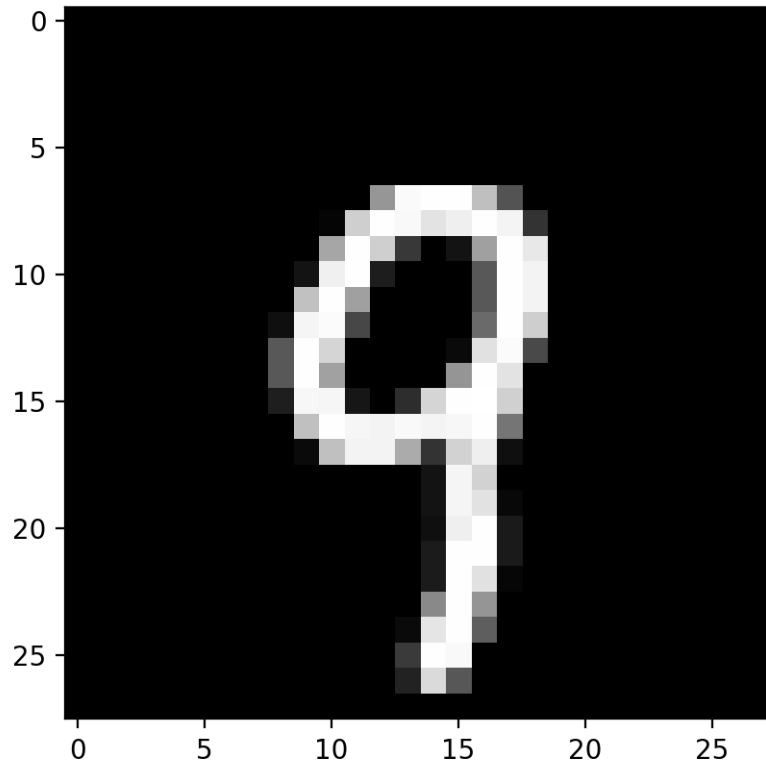
```
[3]: # Ordenamos los datos para tener parejas de imágenes con su respectiva clase
      # Los datos se encuentran en trainloader así que generamos un iterador para
      ↪extraerlos uno por uno
dataiter = iter(trainloader)

# Obtenemos un lote de ejemplos y sus respectivas etiquetas
images, labels = next(dataiter)
```

Es recomendable verificar que estamos cargando bien el conjunto de datos. Así que a continuación imprimiré uno.

```
[4]: plt.imshow(images[1].numpy().squeeze(), cmap='Greys_r');
      images[1].size()
```

```
[4]: torch.Size([1, 28, 28])
```



2.2 Implementación de la red neuronal multicapa

Ahora pasaremos a la creación de la red neuronal, como ejemplo utilizaremos un perceptrón multicapa para clasificar las imágenes del conjunto MNIST. Como entrada tendremos 784 nodos = $28 * 28$, en seguida tendremos una capa oculta de 128 nodos, con una función de activación tipo RELU, después tendremos una segunda capa oculta con 64 nodos y función de activación RELU, en seguida tendremos 10 nodos de salida los cuales pasan por una función softmax que convierte los valores a probabilidades. En el siguiente ejercicio incluiremos la pérdida (loss) con la función de entropía cruzada.

El modulo que contiene las herramientas para crear la RN es **pytorch.nn**. La red neuronal en sí se crea como una clase que hereda la estructura de **pytorch.nn.Module**. Cada una de las capas de la red se define de forma independiente. e.g. Para crear una capa con 784 entradas y 128 nodos utilizamos `nn.Linear(784, 128)`

La red implementa la función *forward* que realiza el paso frontal (forward pass). Esta función miembro recibe un tensor como entrada y calcula la salida de la red.

Varias funciones de activación se encuentran en el módulo `nn.functional`. Dicho módulo usualmente se importa como *F*.

```
[5]: # importamos paquetes de pytorch
from torch import nn
import torch.nn.functional as F
```

En general las redes implementan a partir de la clase `nn.Module` que provee la clase base. Por lo tanto en este ejercicio declararemos una clase denominada red neuronal que hereda de `nn.Module`. Una vez declarada nuestra red neuronal es necesario incluir como atributos de la clase las capas que se requieren, esto por que cada capa incluye los parámetros (pesos) que se entrenarán y deben tener permanencia mientras exista la red. Dichas capas se incluirán dentro del constructor `__init__`.

De acuerdo a pytorch.org

```
nn.Linear(in_features, out_features, bias=True, device=None, dtype=None)
```

Applies a linear transformation to the incoming data.

Por lo tanto si queremos incluir una sola capa podríamos codificar lo siguiente:

```
self.fc = nn.Linear(n_inputs, n_outputs)
```

A continuación definiremos el comportamiento de inferencia de la red dentro de la función `forward`. En el comportamiento indicaremos el orden en el que se van ejecutando las capas y las funciones de activación. Recuerda que las funciones de activación solo se llaman pero no se instancian. Un ejemplo de una sola capa sería:

```
def forward(self, x):
    y = F.relu(self.fc(x))
    return y
```

Al final de la inferencia se debe convertir la salida a probabilidades usando la función `softmax`. Dicha función se encuentra en el módulo `nn.functional` y se llama `F.softmax`. Recuerda que `softmax` recibe como segundo parámetro la dimensión sobre la cual se aplica la función.

```
[6]: # Implementación de la red neuronal
class RedNeuronal(nn.Module):
    def __init__(self):
        super().__init__()
        # TODO: Definir las capas. Cada una con 128, 64 y 10 unidades
        ↪respectivamente
        # 1 punto
        self.fc1 = nn.Linear(784, 128) # 784 es el número de píxeles de cada
        ↪imagen (28*28)
        self.fc2 = nn.Linear(128, 64) # 128 es el número de unidades en la
        ↪primera capa oculta
        self.fc3 = nn.Linear(64, 10) # 64 es el número de unidades en la
        ↪segunda capa oculta

    def forward(self, x):
        ''' Pase frontal de la red, regresamos las probabilidades '''
        # TODO: Define el comportamiento de inferencia, utiliza funciones de
        ↪activación sigmoides
        # y recuerda que al final esta función debe retornar probabilidades y
        ↪no logits.
        # 1 punto
        y = self.fc1(x)
```

```

y = F.sigmoid(y)
y = self.fc2(y)
y = F.sigmoid(y)
y = self.fc3(y)
y = F.softmax(y, dim=1) # Aplicamos softmax para obtener probabilidades

return y

```

```

[7]: model = RedNeuronal()
print(model)

```

```

RedNeuronal(
  (fc1): Linear(in_features=784, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=64, bias=True)
  (fc3): Linear(in_features=64, out_features=10, bias=True)
)

```

2.2.1 Inicializamos pesos y sesgos

Cuando creas las capas se crean también los tensores correspondientes a los pesos y sesgos. Éstos son inicializados por ti, aunque puedes modificarlos usando funciones extra. Para observar sus valores puedes llamar a `model.fc1.weight`

```

[8]: print(model.fc1.weight)
print(model.fc1.bias)

```

Parameter containing:

```

tensor([[[-0.0135,  0.0204,  0.0327, ...,  0.0173, -0.0221, -0.0299],
         [ 0.0195,  0.0341,  0.0289, ..., -0.0157,  0.0294, -0.0011],
         [ 0.0327,  0.0160,  0.0227, ..., -0.0177, -0.0211,  0.0336],
         ...,
         [-0.0221, -0.0044, -0.0226, ...,  0.0078,  0.0028,  0.0336],
         [-0.0039,  0.0100,  0.0334, ...,  0.0352, -0.0188,  0.0320],
         [-0.0255,  0.0157, -0.0213, ...,  0.0220, -0.0258, -0.0285]],
        requires_grad=True)

```

Parameter containing:

```

tensor([ 0.0108,  0.0031, -0.0098,  0.0188, -0.0262,  0.0311,  0.0156, -0.0175,
        -0.0174,  0.0144,  0.0021, -0.0265, -0.0210,  0.0040,  0.0200,  0.0326,
        -0.0151, -0.0080,  0.0170, -0.0319, -0.0017,  0.0191, -0.0233, -0.0013,
        -0.0262, -0.0154,  0.0223,  0.0275, -0.0153, -0.0166, -0.0006, -0.0122,
        -0.0047,  0.0157,  0.0296, -0.0110, -0.0106, -0.0041,  0.0155, -0.0118,
        -0.0285,  0.0063,  0.0248, -0.0003, -0.0015, -0.0281, -0.0128, -0.0040,
        -0.0213,  0.0307, -0.0035,  0.0200, -0.0237,  0.0028, -0.0334,  0.0239,
        -0.0213,  0.0185, -0.0190, -0.0326, -0.0062, -0.0004, -0.0222, -0.0325,
        -0.0337,  0.0174, -0.0302, -0.0013,  0.0006, -0.0020, -0.0186, -0.0186,
        -0.0144, -0.0154, -0.0033, -0.0239, -0.0213, -0.0225,  0.0179, -0.0179,
         0.0116, -0.0316,  0.0213, -0.0287, -0.0314, -0.0025, -0.0279, -0.0084,
         0.0297, -0.0333,  0.0315, -0.0039,  0.0051, -0.0017,  0.0317,  0.0012,
         0.0187, -0.0107, -0.0073,  0.0011,  0.0210, -0.0041, -0.0116, -0.0088,

```

Supongamos que deseamos inicializar los pesos con algunos valores personalizados. Dado que los pesos y sesgos en sí son variables de autograd (Preparadas para el cálculo del gradiente automático) estos solo se pueden modificar cuando no están en modo de autogradiente.

[illegible]

```
[10]: tensor([[-4.9850e-06, -1.0342e-02, -4.2036e-03, ..., 1.1005e-02,
               -9.4923e-03, 2.5289e-03],
              [-6.3974e-03, -7.5546e-04, -4.2238e-03, ..., 2.3753e-03,
               -1.2468e-03, 1.3386e-02],
              [-9.6889e-03, -2.0338e-02, -9.7269e-03, ..., 1.1755e-03,
               -8.7432e-03, -1.3420e-02],
              ...,
              [ 8.3123e-03, -9.9147e-04, -3.4170e-03, ..., -1.0983e-02,
               -3.4648e-03, -7.2444e-03],
              [-1.2603e-02, 2.6075e-02, 1.5074e-02, ..., 1.3766e-03,
               -5.1885e-03, 9.3854e-03],
              [ 1.1063e-02, 1.5157e-02, -6.6482e-03, ..., 1.8667e-02,
               5.6919e-03, -1.5634e-02]])
```

6

```
[11]: # Obtengamos el siguiente lote de imágenes
#dataiter = iter(trainloader)
images, labels = next(dataiter)

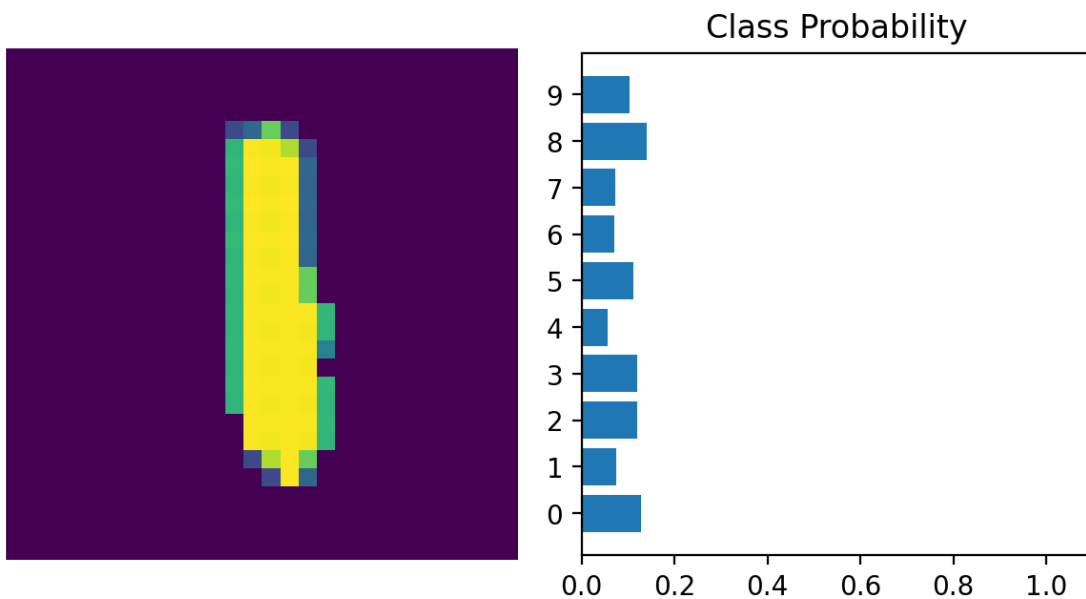
# Reestructuremos el lote a un vector de una dimensión, hay quien le llama a
# esta operación "aplanado".
# La nueva forma será (batch size, color channels, image pixels)
images.resize_(batch_size, 1, 784)
# alternativa: images.resize_(images.shape[0], 1, 784) to not automatically get
# batch size

# Pase frontal de la red
img_idx = 0
prediction = model.forward(images[img_idx,:])

print(prediction)

tensor([[0.1276, 0.0750, 0.1194, 0.1203, 0.0563, 0.1117, 0.0712, 0.0733, 0.1409,
        0.1043]], grad_fn=<SoftmaxBackward0>)
```

```
[12]: img = images[img_idx]
helper.view_classify(img.view(1, 28, 28), prediction)
```



Seguro ninguna de las clases tiene una probabilidad grande con respecto de las otras, esto se debe a que todavía no hemos entrenado la red. En el siguiente ejercicio entrenaremos la red.